



M.Kumarasamy
College of Engineering

Autonomous | Affiliated to Anna University Chennai

Approved by AICTE, Accredited by NBA and NAAC
Thalavapalayam, Karur - 639 113, Tamilnadu, India.



A Project Report

on

DIGITAL CONTACT DIRECTORY

Submitted in partial fulfilment of requirements for the award of the course

of

CGB1121 – DATA STRUCTURES

Under the guidance of

Dr. B. PADMINI DEVI M.E., Ph.D.,

Professor/CSE

Submitted By

HEMASRI V (927625BCS050)

MOUSIYA M (927625BCS098)

NEHA S (927625BIT069)

DEPARTMENT OF FRESHMAN ENGINEERING

M.KUMARASAMY COLLEGE OF ENGINEERING
(Autonomous)

KARUR – 639 113

MAY 2026



M. KUMARASAMY COLLEGE OF ENGINEERING

(Autonomous Institution affiliated to Anna University, Chennai)

KARUR – 639 113

BONAFIDE CERTIFICATE

Certified that this project report on “**DIGITAL CONTACT DIRECTORY**”
is the Bonafide work of **HEMASRI V (927625BCS050)**, **MOUSIYA M (927625BCS098)**, **NEHA S (927625BIT069)** who carried out the project work during the academic year 2025 - 2026 under my supervision.

Signature

Dr. B. PADMINI DEVI, M.E., Ph.D.,

SUPERVISOR,

Department of Computer Science and Engineering,

M. Kumarasamy College of Engineering,

Thalavapalayam, Karur -639 113.

Signature

Dr. K. CHITIRAKALA, M.Sc., M.Phil., Ph.D.,

HEAD OF THE DEPARTMENT,

Department of Freshman Engineering,

M. Kumarasamy College of Engineering,

Thalavapalayam, Karur -639 113.

Submitted for the End Semester Review held on _____

INTERNAL EXAMINER



M.Kumarasamy College of Engineering

Autonomous | Affiliated to Anna University Chennai

Approved by AICTE, Accredited by NBA and NAAC
Thalavapalayam, Karur - 639 113, Tamilnadu, India.



VISION AND MISSION OF THE INSTITUTION

VISION OF THE INSTITUTION

To emerge as a leader among the top institutions in the field of technical education

MISSION OF THE INSTITUTION

- Produce smart technocrats with empirical knowledge who can surmount the global challenges
- Create a diverse, fully-engaged, learner-centric campus environment to provide quality education to the students
- Maintain mutually beneficial partnerships with our alumni, industry, and Professional associations



PROGRAM OUTCOMES (POs)

Engineering students will be able to:

1. PO1: Engineering knowledge: Apply knowledge of mathematics, natural science computing, engineering fundamentals and an engineering specialization to develop to the solution of complex engineering problems.
2. PO2: Problem analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development
3. PO3: Design/development of solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole- life cost, net zero carbon, culture, society and environment as required.
4. PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions
5. PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.
6. PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.
7. PO7: Ethics: Apply ethical principles and commit to professional ethics,



M. Kumarasamy College of Engineering

Autonomous | Affiliated to Anna University Chennai

Approved by AICTE, Accredited by NBA and NAAC
Thalavapalayam, Karur - 639 113, Tamilnadu, India.



human values, diversity and inclusion; adhere to national & international laws.

8. PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
9. PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations
10. PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision- making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments
11. PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change



ABSTRACT

The Phone Directory Application is a contact management system developed using the Doubly Linked List data structure to provide an efficient and flexible way of storing and managing contact information. The application is designed to help users organize their contacts systematically and perform various operations such as adding new contacts, searching for existing records, updating contact details, displaying stored information, and deleting unwanted entries. These features make the system useful for maintaining contact records in an organized and accessible manner. Traditional contact management methods often face challenges in handling frequent modifications and efficient data retrieval. To overcome these limitations, the project utilizes a doubly linked list, which supports dynamic memory allocation and enables efficient insertion, deletion, and bidirectional traversal of records. This allows users to navigate through contact lists easily and manage data more effectively. The application also incorporates exception handling mechanisms to improve system reliability and prevent unexpected program failures. These mechanisms ensure smooth operation by detecting and managing runtime errors. In addition, a simple and user-friendly interface enhances usability and provides a convenient experience for users. This project demonstrates the practical application of data structures in solving real-world problems and highlights the importance of efficient data management, error handling, and user-centered design in the development of reliable and effective software solutions.



ABSTRACT WITH POs MAPPING

ABSTRACT	POs MAPPED
The Phone Directory Application is a contact management system developed using the Doubly Linked List data structure to provide an efficient and flexible way of storing and managing contact information. The application is designed to help users organize their contacts systematically and perform various operations such as adding new contacts, searching for existing records, updating contact details, displaying stored information, and deleting unwanted entries. These features make the system useful for maintaining contact records in an organized and accessible manner. Traditional contact management methods often face challenges in handling frequent modifications and efficient data retrieval. To overcome these limitations, the project utilizes a doubly linked list, which supports dynamic memory allocation and enables efficient insertion, deletion, and bidirectional traversal of records. This allows users to navigate through contact lists easily and manage data more effectively. The application also incorporates exception handling mechanisms to improve system reliability and prevent unexpected program failures. These mechanisms ensure smooth operation by detecting and managing runtime errors. In addition, a simple and user-friendly interface enhances usability and provides a convenient experience for users. This project demonstrates the practical application of data structures in solving real-world problems and highlights the importance of efficient data management, error handling, and user-centered design in the development of reliable and effective software solutions.	PO1(3) PO2(3) PO3(3) PO4(2) PO5(3) PO6(1) PO7(2) PO8(2) PO9(2) PO10(2) PO11(2)

Note: 1- Low, 2-Medium, 3- High



TABLE OF CONTENTS

CHAPTER No.	TITLE	PAGE No.
1	Introduction	
	1.1 Introduction	1
	1.2 Objective	1
	1.3 Data Structure Choice	2
2	Project Methodology	
	2.1 Digital Contact Directory	3
	2.2 Block Diagram	4
3	Modules	
	3.1 Login Module	5
	3.2 Add Contact Module	5
	3.3 View Contact Module	6
	3.4 Search Contact Module	6
	3.5 Update Contact Module	6
	3.6 Delete Contact Module	6
	3.7 Data Storage Module	7
	3.8 User Interference Module	7
4	Results and Discussion	8
5	Conclusion	11
	References	11
	Appendix	12



CHAPTER 1

INTRODUCTION

1.1 Introduction

A phone directory application is an essential tool used to store, manage, and organize contact information in an efficient and structured manner. In today's digital world, where people handle a large number of contacts daily, maintaining them manually becomes difficult. Hence, developing a software-based phone directory helps users easily add, view, update, search, and delete contact details. The phone directory application is implemented using a **doubly linked list** data structure. A doubly linked list consists of nodes, where each node contains contact information along with two pointers: one pointing to the next node and another pointing to the previous node. This bidirectional structure allows users to traverse the list both forward and backward, making navigation flexible and efficient. Using a doubly linked list provides several advantages for this application. It allows dynamic memory allocation, meaning contacts can be added or removed without any fixed size limitation. It also supports efficient insertion and deletion operations compared to other data structures like arrays. Each node in the directory stores details such as the contact's name, phone number, address, and other relevant information, making the system well-organized and easy to manage.

1.2 Objective

To create a system that allows users to add, view, update, and delete contacts easily. To implement bidirectional navigation, enabling users to move forward and backward through the contact list. To provide search functionality for quickly finding contacts based on name or phone number. To support sorting and filtering options, helping users organize contacts in a structured manner. To design a user-friendly interface that ensures ease of use and better user experience. To ensure efficient data handling with minimal time complexity for operations like insertion, deletion, and traversal.



1.3 Data Structure Choice

When considering data structures for storing contact information in a phone directory application, the choice is critical for optimizing operations like insertion, deletion, and search. One commonly compared option is the doubly linked list. Doubly linked lists offer advantages in terms of efficient insertion and deletion at any position, allowing for easy reorganization of contacts. The bidirectional traversal capability enables smooth navigation, supporting functionalities like sorting and iterative processing. However, the drawback lies in the additional memory required to store two pointers per node, potentially impacting space efficiency.



CHAPTER 2

PROJECT METHODOLOGY

2.1 Digital Contact Directory

A Digital Contact Directory is a software application designed to store, manage, and organize contact information in a structured digital format. It allows users to maintain important details such as names, phone numbers, email addresses, and other relevant information in a centralized system. Unlike manual record-keeping methods, this system provides fast access, easy updates, and efficient management of contacts through a user-friendly interface.

Key Features of a Digital Contact Directory:

1. **Centralized Storage:**

All contact information is stored in a single digital database, making it easy to access and manage from one place.

2. **Add Contacts:**

Users can easily add new contact details including name, phone number, email, and other personal or professional information.

3. **Search Functionality:**

The system allows quick searching of contacts using keywords like name or number, ensuring fast retrieval of information.

4. **Update Records:**

Existing contact details can be modified or updated whenever required to keep information accurate and current.

5. **Delete Contacts:**

Unnecessary or outdated contacts can be removed from the system to maintain a clean and organized database..

6. **User-Friendly Interface:**

The application provides a simple and easy-to-use interface that allows users to manage contacts without technical difficulty.

7. **Efficient Data Management:**

It reduces manual effort, avoids duplication, and ensures better organization and reliability of contact information.



2.2 Block Diagram

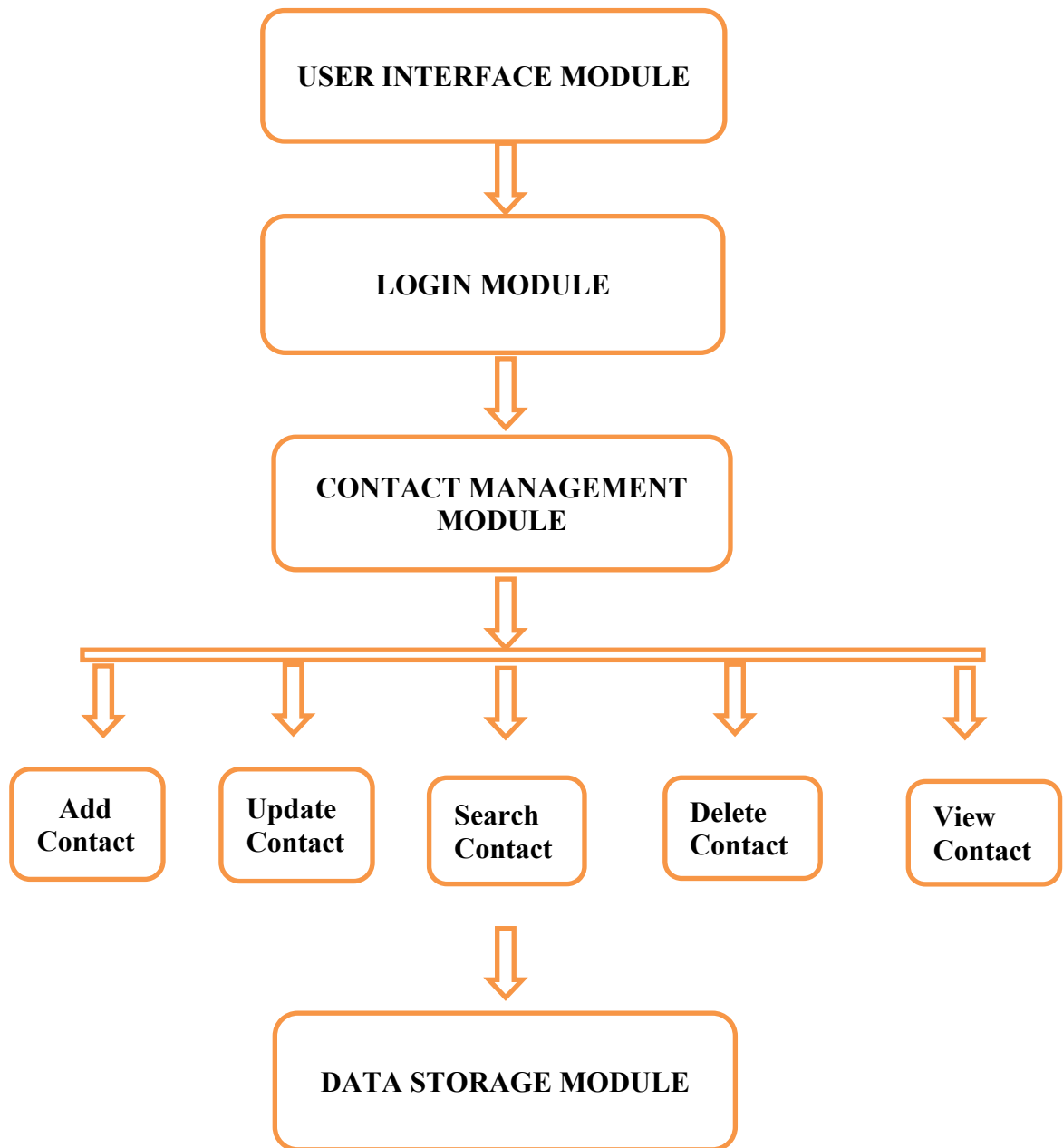


Fig.2.2.1 Block Diagram



CHAPTER 3

MODULES

The following modules are implemented in the Direct Contact Dictionary:

1. Login Module
2. Add Contact Module
3. View Contact Module
4. Search Contact Module
5. Update Contact Module
6. Delete Contact Module
7. Data Storage Module
8. User Interference Module

3.1. LOGIN MODULE

The Login Module is responsible for providing secure access to the system. It allows the user to enter a username and password, which are then verified against stored credentials. If the entered details match, the user is granted access to the system; otherwise, access is denied. This module ensures that only authorized users can use the application and protects the data from unauthorized access.

3.2. ADD CONTACT MODULE

The Add Contact Module is used to create and store new contact information. In this module, the user enters details such as name, phone number, email address, etc. The system may validate the input to ensure correctness (for example, checking if fields are not empty). Once validated, the contact details are stored in the system's data storage. This module is essential for building the contact database.

3.3. VIEW CONTACT MODULE

The View Contact Module is used to display all stored contacts. It retrieves the contact data from storage and presents it in an organized format, such as a list or table. This module helps users easily access and review all saved contacts in one place, improving usability and visibility of data.



3.4. SEARCH CONTACT MODULE

The Search Contact Module allows users to locate specific contacts quickly. The user can search using keywords such as a name or phone number. The system scans through the stored data and displays matching results. This module improves efficiency by reducing the time required to find a particular contact.

3.5. UPDATE CONTACT MODULE

The Update Contact Module is responsible for modifying existing contact details. Users can select a contact and update its information, such as changing a phone number or correcting a name. The system replaces the old data with the updated data. This module ensures that the contact information remains accurate and up to date.

3.6. DELETE CONTACT MODULE

The Delete Contact Module is used to remove unwanted or outdated contacts from the system. The user selects a contact to delete, and the system permanently removes it from storage. This helps maintain a clean and organized database by eliminating unnecessary records.

3.7. DATA STORAGE MODULE

The Data Storage Module manages how contact information is stored and maintained internally. It typically uses data structures such as arrays or lists to store contact records. This module supports operations like adding, retrieving, updating, and deleting data. It acts as the backend of the system, ensuring efficient data handling and management.

3.8. USER INTERFERENCE MODULE

The Data Storage Module manages how contact information is stored and maintained internally. It typically uses data structures such as arrays or lists to store contact records. This module supports operations like adding, retrieving, updating, and deleting data. It acts as the backend of the system, ensuring efficient data handling and management.



CHAPTER 4

RESULTS AND DISCUSSION

4.1 Results

4.1.1 Add Contacts

Fig 4.1.1 Add contact

4.1.2 Search Contacts

Fig 4.1.2 Search Contact



4.1.3 Update Contacts

Editing contact

FULL NAME

hemasri

PHONE

95698696566

EMAIL

hema@gmail.com

✓ Update Contact

Cancel

Fig 4.1.3 Update Contact

4.1.4 Delete Contacts

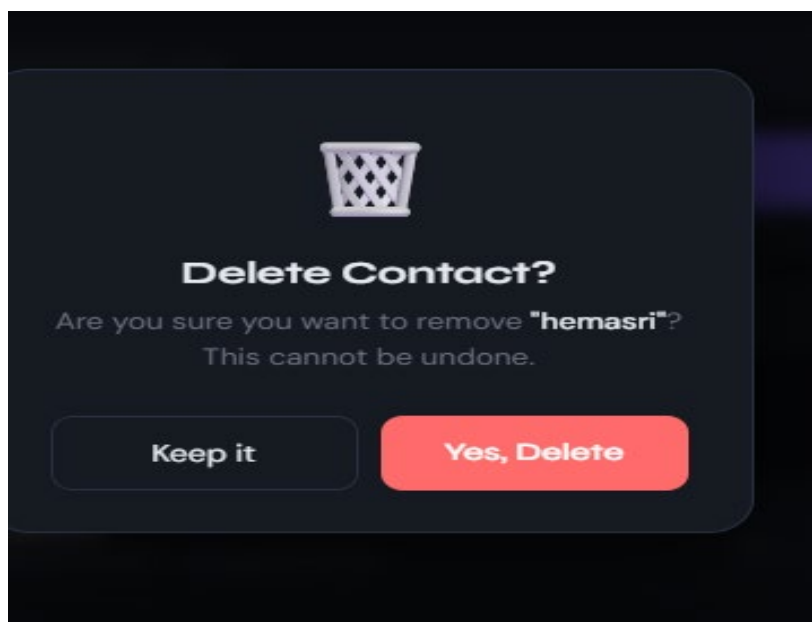


Fig 4.1.4 Delete Contact



4.2 Discussion

The phone directory application, implemented using a doubly linked list, serves as a practical and dynamic solution for managing contacts efficiently. The choice of a doubly linked list allows for seamless insertion, deletion, and bidirectional traversal, essential for a user-friendly contact management system. The application encompasses key functionalities, including adding, updating, and deleting contacts, ensuring the integrity of the contact directory. Exception handling enhances the robustness of the system by addressing unforeseen errors, preventing runtime failures, and enabling graceful recovery. The user interface provides a clear and intuitive experience, displaying contact information and allowing users to search, edit, and delete contacts seamlessly. This project not only showcases proficiency in data structure implementation but also emphasizes the importance of user-centric design and exception handling for creating a reliable and user-friendly phone directory application.



CHAPTER 5

CONCLUSION

The system provides a simple and efficient solution for managing personal contact information through a user-based approach. It ensures secure access by allowing users to register and log in with their credentials. Each user can manage their own contacts independently, with features such as adding, viewing, searching, updating, sorting, and deleting contacts. The system maintains data privacy by ensuring that no user can access another user's information. Overall, it offers a user-friendly interface and organized data management, making it a reliable and practical tool for handling contact details. In addition, the system is designed with simplicity and scalability in mind, making it easy to extend with advanced features such as cloud storage, mobile access, or database integration. It demonstrates the effective use of web technologies to create an interactive and responsive application. This project also highlights the importance of authentication, data handling, and user experience in modern applications. With further enhancements, the system can be developed into a full-fledged contact management platform suitable for real-world use.



M. Kumarasamy College of Engineering

Autonomous | Affiliated to Anna University Chennai

Approved by AICTE, Accredited by NBA and NAAC
Thalavapalayam, Karur - 639 113, Tamilnadu, India.



REFERENCES

1. On the Use of Model Checking for the Bounded and Unbounded Verification of Nonblocking Concurrent Data Structures | Science Gate
2. c - Creating a phonebook using linked list and file operations - Stack Overflow
3. phone-directory/Datastructureprojcet.pdf at main · cloner plus/phone-directory · GitHub
 - [MDN Web Docs](#) – HTML, CSS, JavaScript documentation and examples.
 - [W3Schools](#) – HTML, CSS, JavaScript tutorials and references.
 - [Google Fonts](#) – Syne and DM Sans fonts used in the project.
 - [MDN JavaScript Guide](#) – JavaScript concepts such as DOM manipulation, Local



APPENDIX

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>ContactVault</title>
<link
href="https://fonts.googleapis.com/css2?family=Syne:wght@400;600;700;800&family=DM+Sans:ital,wght@0,300;0,400;0,500;1,300&display=swap" rel="stylesheet">

<style>
:root {
  --bg: #0d0f14;
  --surface: #161a22;
  --surface2: #1e2330;
  --border: #2a3045;
  --accent: #7b61ff;
  --accent2: #ff6b6b;
  --accent3: #00d4aa;
  --text: #e8eaf0;
  --muted: #6b7280;
  --card-glow: rgba(123, 97, 255, 0.08);
}

* { box-sizing: border-box; margin: 0; padding: 0; }

body {
  font-family: 'DM Sans', sans-serif;
```



```
background: var(--bg);
color: var(--text);
min-height: 100vh;
overflow-x: hidden;
}
body::before {
  content: "";
  position: fixed;
  top: -200px;
  right: -200px;
  width: 600px;
  height: 600px;
  background: radial-gradient(circle, rgba(123,97,255,0.12) 0%, transparent 70%);
  pointer-events: none;
  z-index: 0;
}
body::after {
  content: "";
  position: fixed;
  bottom: -200px;
  left: -200px;
  width: 500px;
  height: 500px;
  background: radial-gradient(circle, rgba(0,212,170,0.08) 0%, transparent 70%);
  pointer-events: none;
  z-index: 0;
}
.page {
  position: relative;
```



```
z-index: 1;
max-width: 520px;
margin: 0 auto;
padding: 40px 20px;
}
min-height: 100vh;
/* — LOGO — */
.logo {
  font-family: 'Syne', sans-serif;
  font-weight: 800;
  font-size: 1.6rem;
  letter-spacing: -0.5px;
  display: flex;
  align-items: center;
  gap: 8px;
  margin-bottom: 40px;
}

.logo span { color: var(--accent); }
.logo-dot { width: 8px; height: 8px; background: var(--accent3); border-radius: 50%; margin-left: 2px; display: inline-block; animation: pulse 2s infinite; }

@keyframes pulse {
  0%, 100% { opacity: 1; transform: scale(1); }
  50% { opacity: 0.5; transform: scale(0.8); }
}

/* — AUTH — */
#authBox { animation: fadeUp 0.5s ease; }
#appBox { animation: fadeUp 0.4s ease; }

@keyframes fadeUp {
```



```
from { opacity: 0; transform: translateY(20px); }  
to { opacity: 1; transform: translateY(0); }  
}
```

```
.auth-header {  
  margin-bottom: 32px;  
}  
.auth-header h1 {  
  font-family: 'Syne', sans-serif;  
  font-weight: 800;  
  font-size: 2.4rem;  
  line-height: 1.1;  
  margin-bottom: 8px;  
}
```

```
.auth-header h1 em {  
  font-style: normal;  
  color: var(--accent);  
}
```

```
.auth-header p {  
  color: var(--muted);  
  font-size: 0.95rem;  
}
```

```
.card {  
  background: var(--surface);  
  border: 1px solid var(--border);  
  border-radius: 16px;
```



```
padding: 24px;
margin-bottom: 12px;
}
.field {
  position: relative;
  margin-bottom: 14px;
}
.field label {
  display: block;

font-size: 0.75rem;
font-weight: 500;
color: var(--muted);
text-transform: uppercase;
letter-spacing: 0.8px;
margin-bottom: 6px;
}

.field input {
  width: 100%;
  background: var(--surface2);
  border: 1px solid var(--border);
  border-radius: 10px;
  padding: 12px 14px;
  color: var(--text);
  font-family: 'DM Sans', sans-serif;
  font-size: 0.95rem;
  transition: border-color 0.2s, box-shadow 0.2s;
  outline: none;
}
```




```
.field input:focus {  
  border-color: var(--accent);  
  
  box-shadow: 0 0 0 3px rgba(123,97,255,0.15);  
}
```

```
.field input::placeholder { color: var(--muted); }
```

```
.btn-row {  
  display: grid;  
  grid-template-columns: 1fr 1fr;  
  gap: 10px;  
  margin-top: 4px;  
}  
  
.btn {  
  padding: 12px;  
  border: none;  
  border-radius: 10px;  
  font-family: 'DM Sans', sans-serif;  
  font-size: 0.9rem;  
  font-weight: 500;  
  cursor: pointer;  
  transition: all 0.2s;  
  letter-spacing: 0.2px;  
}
```

```
.btn-primary {  
  background: var(--accent);  
  color: white;  
}
```



```
.btn-primary:hover {  
  background: #6a4ff5;  
  transform: translateY(-1px);  
  box-shadow: 0 4px 20px rgba(123,97,255,0.4);  
}
```

```
.btn-secondary {  
  background: var(--surface2);  
  color: var(--text);  
  border: 1px solid var(--border);  
}
```

```
.btn-secondary:hover {  
  border-color: var(--accent);  
  color: var(--accent);  
}
```

```
.top-bar {
```

```
/* — TOP BAR — */
```

```
display: flex;  
align-items: center;  
justify-content: space-between;  
margin-bottom: 28px;  
}
```

```
.user-chip {  
  display: flex;  
  align-items: center;  
  gap: 10px;  
  background: var(--surface);
```



```
border: 1px solid var(--border);
```

```
padding: 8px 14px;
```

```
border-radius: 50px;
```

```
font-size: 0.85rem;
```

```
}
```

```
.avatar {
```

```
width: 28px;
```

```
height: 28px;
```

```
border-radius: 50%;
```

```
background: linear-gradient(135deg, var(--accent), var(--accent3));
```

```
display: flex;
```

```
align-items: center;
```

```
justify-content: center;
```

```
font-family: 'Syne', sans-serif;
```

```
font-size: 0.75rem;
```

```
font-weight: 700;
```

```
color: white;
```

```
}
```

```
.btn-logout {
```

```
background: transparent;
```

```
border: 1px solid var(--border);
```

```
color: var(--muted);
```

```
padding: 8px 14px;
```

```
border-radius: 50px;
```

```
font-size: 0.8rem;
```

```
cursor: pointer;
```

```
transition: all 0.2s;
```

```
font-family: 'DM Sans', sans-serif;
```

```
}
```



```
.btn-logout:hover {  
  border-color: var(--accent2);  
  color: var(--accent2);  
}  
  
/* — SECTION TITLE — */  
.section-title {  
  font-family: 'Syne', sans-serif;  
  font-size: 0.7rem;  
  font-weight: 700;  
  text-transform: uppercase;  
  letter-spacing: 1.5px;  
  color: var(--muted);  
  margin-bottom: 10px;  
}  
  
/* — SEARCH — */  
.search-wrap {  
  position: relative;  
  margin-bottom: 12px;  
}  
  
.search-icon {  
  position: absolute;  
  left: 14px;  
  top: 50%;  
  transform: translateY(-50%);  
  color: var(--muted);  
  font-size: 1rem;  
}  
.search-wrap input {
```



```
width: 100%;
background: var(--surface);
border: 1px solid var(--border);
border-radius: 10px;
padding: 12px 14px 12px 40px;
color: var(--text);
font-family: 'DM Sans', sans-serif;
font-size: 0.9rem;
outline: none;
transition: border-color 0.2s;
}

.search-wrap input:focus { border-color: var(--accent); }
.search-wrap input::placeholder { color: var(--muted); }

/* — CONTACT CARDS — */
.contact-list { display: flex; flex-direction: column; gap: 8px; }

.contact-card {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 12px;
  padding: 16px;
  display: flex;
  align-items: center;
  gap: 14px;
  transition: all 0.2s;
  animation: slideIn 0.3s ease;
}
```



```
@keyframes slideIn {
  from { opacity: 0; transform: translateX(-10px); }
  to { opacity: 1; transform: translateX(0); }
}

.contact-card:hover {
  border-color: var(--border);
  background: var(--surface2);
  box-shadow: 0 4px 24px var(--card-glow);
}

.contact-avatar {
  width: 42px;
  height: 42px;
  border-radius: 12px;
  background: linear-gradient(135deg, var(--accent), #b06aff);
  display: flex;
  align-items: center;
  justify-content: center;
  font-family: 'Syne', sans-serif;
  font-weight: 700;
  font-size: 1rem;
  color: white;
  flex-shrink: 0;
}

.contact-info { flex: 1; min-width: 0; }

.contact-name {
  font-family: 'Syne', sans-serif;
```



```
font-weight: 600;
font-size: 0.95rem;
margin-bottom: 2px;
white-space: nowrap;
overflow: hidden;
text-overflow: ellipsis;
}

.contact-detail {
font-size: 0.78rem;
color: var(--muted);
white-space: nowrap;
overflow: hidden;
text-overflow: ellipsis;
}

.contact-actions { display: flex; gap: 6px; flex-shrink: 0; }

.action-btn {
width: 32px;
height: 32px;
border-radius: 8px;
border: 1px solid var(--border);
background: transparent;
color: var(--muted);
cursor: pointer;
display: flex;
align-items: center;
justify-content: center;
font-size: 0.85rem;
transition: all 0.2s;
```



}

```
.action-btn.edit:hover { background: rgba(123,97,255,0.15); color: var(--accent); border-color: var(--accent); }
```

```
.action-btn.del:hover { background: rgba(255,107,107,0.15); color: var(--accent2); border-color: var(--accent2); }
```

```
/* — EMPTY STATE — */
```

```
.empty-state {  
  text-align: center;  
  padding: 48px 20px;  
  color: var(--muted);  
}
```

```
.empty-state .icon { font-size: 2.5rem; margin-bottom: 12px; }
```

```
.empty-state p { font-size: 0.9rem; }
```

```
/* — STATS — */
```

```
.stats-row {  
  display: grid;  
  grid-template-columns: 1fr 1fr;  
  gap: 10px;  
  margin-bottom: 16px;  
}
```

```
.stat-chip {  
  background: var(--surface);  
  border: 1px solid var(--border);  
  border-radius: 10px;  
  padding: 12px 14px;  
  display: flex;
```




```
align-items: center;  
gap: 10px;  
}
```

```
.stat-icon { font-size: 1.2rem; }
```

22

```
.stat-label { font-size: 0.7rem; color: var(--muted); text-transform: uppercase; letter-spacing:  
0.5px; }
```

```
.stat-value { font-family: 'Syne', sans-serif; font-weight: 700; font-size: 1.1rem; color: var(--  
accent3); }
```

```
/* — TOAST — */
```

```
.toast {  
  position: fixed;  
  bottom: 24px;  
  right: 24px;  
  background: var(--surface2);  
  border: 1px solid var(--border);  
  border-radius: 12px;  
  padding: 12px 18px;  
  font-size: 0.85rem;  
  display: flex;  
  align-items: center;  
  gap: 10px;  
  z-index: 1000;  
  transform: translateY(80px);  
  opacity: 0;  
  transition: all 0.3s cubic-bezier(0.34, 1.56, 0.64, 1);  
  border-left: 3px solid var(--accent3);  
}
```

25



```
.toast.show { transform: translateY(0); opacity: 1; }  
.toast.error { border-left-color: var(--accent2); }
```

```
/* — HIDDEN — */
```

```
.hidden { display: none !important; }
```

```
/* — EDIT MODE — */
```

```
.form-card.editing {  
  border-color: var(--accent);  
  box-shadow: 0 0 0 3px rgba(123,97,255,0.12);  
}
```

```
.edit-badge {  
  display: inline-flex;  
  align-items: center;  
  gap: 6px;  
  background: rgba(123,97,255,0.15);  
  color: var(--accent);  
  border-radius: 6px;  
  padding: 3px 10px;  
  font-size: 0.75rem;  
  font-weight: 500;  
  margin-bottom: 14px;  
}
```

```
.btn-add {  
  width: 100%;  
  background: var(--accent);  
  color: white;  
  padding: 13px;  
  border-radius: 10px;  
  border: none;
```



```
font-family: 'Syne', sans-serif;
font-weight: 600;
font-size: 0.9rem;
letter-spacing: 0.3px;
cursor: pointer;
transition: all 0.2s;
margin-top: 4px;
}

.btn-add:hover {
  background: #6a4ff5;
transform: translateY(-1px);
  box-shadow: 0 6px 20px rgba(123,97,255,0.4);
}

.btn-cancel {
  width: 100%;
  background: transparent;
  color: var(--muted);
  padding: 10px;
  border: none;
  font-family: 'DM Sans', sans-serif;
  font-size: 0.85rem;
  cursor: pointer;
  margin-top: 6px;
  border-radius: 8px;
  transition: color 0.2s;
}

.btn-cancel:hover { color: var(--accent2); }
```



/* — SORT ROW — */

```
.sort-row {  
  display: flex;  
  align-items: center;  
  gap: 6px;  
  margin-top: 12px;  
  flex-wrap: wrap;  
}  
  
.sort-label {  
  font-size: 0.72rem;  
  color: var(--muted);  
  text-transform: uppercase;  
  letter-spacing: 0.8px;  
  font-weight: 500;  
  margin-right: 2px;  
}  
  
.sort-btn {  
  padding: 5px 12px;  
  border-radius: 20px;  
  border: 1px solid var(--border);  
  background: transparent;  
  color: var(--muted);  
  font-family: 'DM Sans', sans-serif;  
  font-size: 0.78rem;  
  cursor: pointer;  
  transition: all 0.2s;  
}  
  
.sort-btn:hover {
```



border-color: var(--accent);

color: var(--accent);

}

.sort-btn.active {

background: var(--accent);

border-color: var(--accent);

color: white;

font-weight: 500;

}

.modal-overlay {

position: fixed;

inset: 0;

background: rgba(0,0,0,0.7);

backdrop-filter: blur(6px);

display: flex;

align-items: center;

justify-content: center;

z-index: 999;

opacity: 0;

pointer-events: none;

transition: opacity 0.25s;

}

.modal-overlay.show {

opacity: 1;

pointer-events: all;

}

.modal-box {

background: var(--surface);

border: 1px solid var(--border);

border-radius: 20px;



```
padding: 32px 28px 24px;
max-width: 340px;
width: 90%;
text-align: center;
transform: scale(0.85) translateY(20px);
transition: transform 0.3s cubic-bezier(0.34,1.56,0.64,1);
box-shadow: 0 24px 60px rgba(0,0,0,0.5);
}

.modal-overlay.show .modal-box {
  transform: scale(1) translateY(0);
}

.modal-icon {
  font-size: 2.8rem;
  margin-bottom: 14px;
  animation: shake 0.5s ease 0.2s both;
}

@keyframes shake {
  0%,100% { transform: rotate(0deg); }
  25% { transform: rotate(-10deg); }
  75% { transform: rotate(10deg); }
}

.modal-title {
  font-family: 'Syne', sans-serif;
  font-weight: 700;
  font-size: 1.15rem;
  margin-bottom: 8px;
}

.modal-sub {
  color: var(--muted);
  font-size: 0.85rem;
```



```
margin-bottom: 24px;
line-height: 1.5;
}
.modal-name {
  color: var(--text);
  font-weight: 500;
}
.modal-btns {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 10px;
}
.modal-keep {
  padding: 12px;
  border-radius: 10px;
  border: 1px solid var(--border);
  background: transparent;
  color: var(--text);
  font-family: 'DM Sans', sans-serif;
  font-size: 0.9rem;
  cursor: pointer;
  transition: all 0.2s;
}
.modal-keep:hover {
  border-color: var(--accent);
  color: var(--accent);
}
.modal-confirm {
  padding: 12px;
  border-radius: 10px;
  border: none;
```



```
background: var(--accent2);
color: white;
font-family: 'Syne', sans-serif;
font-weight: 600;
font-size: 0.9rem;
cursor: pointer;
transition: all 0.2s;
}

.modal-confirm:hover {
  background: #e85555;
  box-shadow: 0 4px 16px rgba(255,107,107,0.4);
}

/* — CARD DELETE ANIMATION — */
.contact-card.deleting {
  animation: cardOut 0.35s ease forwards;
}

@keyframes cardOut {
  0% { opacity: 1; transform: translateX(0) scale(1); max-height: 100px; margin-bottom: 8px; }
  60% { opacity: 0; transform: translateX(60px) scale(0.95); max-height: 100px; }
  100% { opacity: 0; max-height: 0; margin-bottom: 0; padding: 0; }
}

</style>
</head>

<body>
<!-- AUTH -->
<div class="page" id="authBox">
  <div class="logo">Contact<span>Vault</span><span class="logo-dot"></span></div>

  <div class="auth-header">
```




<h1>Your contacts,
organised.</h1>

<p>Sign in or create an account to manage your directory.</p>

</div>

<div class="card">

<div class="field">

<label>Username</label>

<input type="text" id="username" placeholder="e.g. john_doe" autocomplete="off">

</div>

<div class="field">

<label>Password</label>

<input type="password" id="password" placeholder=".....">

</div>

<div class="btn-row">

<button class="btn btn-secondary" onclick="register()">Register</button>

<button class="btn btn-primary" onclick="login()">Login →</button>

</div>

</div>

</div>

<!-- APP -->

<div class="page hidden" id="appBox">

<div class="logo">ContactVault</div>

<div class="top-bar">

<div class="user-chip">

<div class="avatar" id="avatarInitial">?</div>

</div>

<button class="btn-logout" onclick="logout()">Sign out</button>



</div>

<!-- STATS -->

<div class="stats-row">

<div class="stat-chip">

<div>

<div class="stat-label">Contacts</div>

<div class="stat-value" id="statTotal">0</div>

</div>

</div>

<div class="stat-chip">

<div>

<div class="stat-label">Showing</div>

<div class="stat-value" id="statShowing">0</div>

</div>

</div>

</div>

<!-- ADD / EDIT FORM -->

<div class="card form-card" id="formCard" style="margin-bottom:12px">

<div class="edit-badge hidden" id="editBadge">  Editing contact</div>

<div class="field">

<label>Full Name</label>

<input type="text" id="cName" placeholder="Jane Smith">

</div>

<div class="field">

<label>Phone</label>

<input type="text" id="cPhone" placeholder="+91 98765 43210">

</div>



<div class="field">

<label>Email</label>

<input type="text" id="cEmail" placeholder="jane@example.com">

</div>

<button class="btn-add" id="submitBtn" onclick="submitContact()">+ Add

Contact</button>

<button class="btn-cancel hidden" id="cancelBtn" onclick="cancelEdit()">Cancel</button>

</div>

<!-- SEARCH -->

<div class="search-wrap">

🔍

<input type="text" id="search" placeholder="Search by name, phone or email..."

oninput="render()">

</div>

<!-- SORT -->

<div class="sort-row">

Sort by

<button class="sort-btn active" id="sortName" onclick="setSort('name')">A → Z</button>

<button class="sort-btn" id="sortNameDesc" onclick="setSort('nameDesc')">Z →

A</button>

<button class="sort-btn" id="sortNewest" onclick="setSort('newest')">Newest</button>

<button class="sort-btn" id="sortOldest" onclick="setSort('oldest')">Oldest</button>

</div>

<div class="section-title" style="margin-top:14px">Directory</div>

<!-- LIST -->

<div class="contact-list" id="list"></div>



</div>

<!-- DELETE MODAL -->

<div class="modal-overlay" id="deleteModal">

<div class="modal-box">

<div class="modal-icon">  </div>

<div class="modal-title">Delete Contact?</div>

<div class="modal-sub">Are you sure you want to remove ? This cannot be undone.</div>

<div class="modal-btns">

<button class="modal-keep" onclick="closeDeleteModal()">Keep it</button>

<button class="modal-confirm" onclick="confirmDelete()">Yes, Delete</button>

</div>

</div>

</div>

<!-- TOAST -->

<div class="toast" id="toast">

✓

Done

</div>

<script>

let currentUser = null;

let contacts = [];

let editIndex = -1;

/* _____ TOAST _____ */

function toast(msg, error = false) {

const t = document.getElementById('toast');



```
document.getElementById('toastMsg').textContent = msg;

document.getElementById('toastIcon').textContent = error ? '✖' : '✓';

t.classList.toggle('error', error);
t.classList.add('show');
setTimeout(() => t.classList.remove('show'), 2800);
}

/* _____ AUTH _____ */

function register() {
  const u = document.getElementById('username').value.trim();
  const p = document.getElementById('password').value.trim();
  if (!u || !p) return toast('Please fill in both fields', true);
  if (localStorage.getItem('user_' + u)) return toast('Username already taken', true);
  localStorage.setItem('user_' + u, p);
  toast('Account created! You can now log in.');
```

```
}

function login() {
  const u = document.getElementById('username').value.trim();
  const p = document.getElementById('password').value.trim();
  if (!u || !p) return toast('Enter your credentials', true);
  const stored = localStorage.getItem('user_' + u);
  if (stored === p) {
    currentUser = u;
    startApp();
  } else {
    toast('Incorrect username or password', true);
  }
}
```



```
function logout() {
  currentUser = null;
  location.reload();
}

/* ----- APP START ----- */

function startApp() {
  document.getElementById('authBox').classList.add('hidden');
  document.getElementById('appBox').classList.remove('hidden');
  document.getElementById('welcome').textContent = currentUser;
  document.getElementById('avatarInitial').textContent = currentUser[0].toUpperCase();
  load();
  render();
}

/* ----- DATA ----- */

function load() {
  contacts = JSON.parse(localStorage.getItem('contacts_' + currentUser) || '[]');
}

function save() {
  localStorage.setItem('contacts_' + currentUser, JSON.stringify(contacts));
}

/* ----- CONTACT ACTIONS ----- */

function submitContact() {
  const n = document.getElementById('cName').value.trim();
  const ph = document.getElementById('cPhone').value.trim();
  const em = document.getElementById('cEmail').value.trim();
  if (!n || !ph || !em) return toast('Please fill in all fields', true);
```



```
if (editIndex >= 0) {  
    contacts[editIndex] = { name: n, phone: ph, email: em, added: contacts[editIndex].added };  
    toast('Contact updated');  
    cancelEdit();  
} else {  
    contacts.push({ name: n, phone: ph, email: em, added: Date.now() });  
    toast('Contact added');  
    document.getElementById('cName').value = "";  
    document.getElementById('cPhone').value = "";  
    document.getElementById('cEmail').value = "";  
}  
save();  
render();  
}
```

```
function editContact(i) {  
    const c = contacts[i];  
    document.getElementById('cName').value = c.name;  
    document.getElementById('cPhone').value = c.phone;  
    document.getElementById('cEmail').value = c.email;  
    editIndex = i;  
    document.getElementById('submitBtn').textContent = '✓ Update Contact';  
    document.getElementById('editBadge').classList.remove('hidden');  
    document.getElementById('cancelBtn').classList.remove('hidden');  
    document.getElementById('formCard').classList.add('editing');  
    document.getElementById('cName').focus();  
}
```

```
function cancelEdit() {  
    editIndex = -1;  
    document.getElementById('cName').value = "";
```



```
document.getElementById('cPhone').value = "";
document.getElementById('cEmail').value = "";
document.getElementById('submitBtn').textContent = '+ Add Contact';
document.getElementById('editBadge').classList.add('hidden');
document.getElementById('cancelBtn').classList.add('hidden');
document.getElementById('formCard').classList.remove('editing');
}

let currentSort = 'name';

function setSort(type) {
    currentSort = type;
    document.querySelectorAll('.sort-btn').forEach(b => b.classList.remove('active'));
    document.getElementById('sort' + type.charAt(0).toUpperCase() +
type.slice(1)).classList.add('active');
    render();
}

/* — DELETE WITH MODAL — */
let deleteTargetIndex = -1;

function deleteContact(i) {
    deleteTargetIndex = i;
    document.getElementById('modalName').textContent = "" + contacts[i].name + "";
    document.getElementById('deleteModal').classList.add('show');
}

function closeDeleteModal() {
    document.getElementById('deleteModal').classList.remove('show');
    deleteTargetIndex = -1;
}
```




```
function confirmDelete() {
  if (deleteTargetIndex < 0) return;
  const name = contacts[deleteTargetIndex].name;
  const cardId = 'card-' + deleteTargetIndex;
  closeDeleteModal();

  const card = document.getElementById(cardId);
  if (card) {
    card.classList.add('deleting');
    setTimeout(() => {
      contacts.splice(deleteTargetIndex, 1);
      deleteTargetIndex = -1;
      save();
      render();
    }, 340);
  } else {
    contacts.splice(deleteTargetIndex, 1);
    deleteTargetIndex = -1;
    save();
    render();
    toast(name + ' removed');
  }
}

/* _____ RENDER _____ */

function render() {
  const q = document.getElementById('search').value.toLowerCase();
  const filtered = contacts.filter(c =>
    c.name.toLowerCase().includes(q) ||
    c.phone.toLowerCase().includes(q) ||
    c.email.toLowerCase().includes(q)
  )
}
```



```
);  
document.getElementById('statTotal').textContent = contacts.length;  
document.getElementById('statShowing').textContent = filtered.length;  
  
const list = document.getElementById('list');  
  
if (filtered.length === 0) {  
  list.innerHTML = `  
    <div class="empty-state">  
      <div class="icon">${contacts.length === 0 ? '📋' : '💬'}</div>  
      <p>${contacts.length === 0 ? 'No contacts yet. Add your first one above.' : 'No contacts  
match your search.'}</p>  
    </div>`;   
  return;  
}  
// Sort  
let sorted = [...filtered];  
if (currentSort === 'name') sorted.sort((a,b) => a.name.localeCompare(b.name));  
else if (currentSort === 'nameDesc') sorted.sort((a,b) => b.name.localeCompare(a.name));  
else if (currentSort === 'newest') sorted.sort((a,b) => (b.added||0) - (a.added||0));  
else if (currentSort === 'oldest') sorted.sort((a,b) => (a.added||0) - (b.added||0));  
  
list.innerHTML = sorted.map((c, i) => {  
  const realIndex = contacts.indexOf(c);  
  const initials = c.name.split(' ').map(w => w[0]).join('').slice(0,2).toUpperCase();  
  const colors = ['#7b61ff','#b06aff','#ff6b6b','#ff9a5c','#00d4aa','#00a3ff','#f7c948','#ff9a5c'];  
  const grad = colors[i % colors.length];  
  return `  
    <div class="contact-card" id="card-${realIndex}">  
      <div class="contact-avatar" style="background:linear-
```



gradient(135deg,\$\{\text{grad}\})">\\$\{\text{initials}\}</div>

<div class="contact-info">

<div class="contact-name">\\$\{c.name\}</div>

<div class="contact-detail">\\$\{c.phone\} · \\$\{c.email\}</div>

</div>

<div class="contact-actions">

<button class="action-btn edit" onclick="editContact(\\$\{realIndex\})"

title="Edit">\</button>

<button class="action-btn del" onclick="deleteContact(\\$\{realIndex\})"

title="Delete">\</button>

</div>

</div>`;

}).join("");

}

</script>

</body>

</html>